MiniLex

Visual Lexical Analyzer

Compiler Design Project | PS-01

Guide: UMA Shankari

Team Members

2420030540 – Allipuram Akhil
2420030563 – B. Harneesh Yadav
2420030639 – A. Dhanush

Problem Statement

Why an interactive lexical analysis workbench is needed

- Students learn regular expressions and finite automata, but often find it difficult to connect the concepts to actual compiler behavior.
- Manual inspection of source code makes token identification, lexeme boundaries and lexical errors difficult to track.
- Without visual feedback, understanding how NFA and DFA recognize token patterns can be challenging.
- MiniLex aims to make lexical analysis transparent by showing token output, automata construction and acceptance decisions.

Project Objectives

What MiniLex is designed to achieve

- Build a browser-based workbench for analyzing source code written in a small C-like language.
- Allow users to create, edit and validate token rules using regular expressions.
- Generate a token stream with token names, lexemes, line numbers and highlighted state transitions.
- Construct and compare NFA and DFA for selected token rules.
- Detect invalid characters and lexical errors with their source locations.

System Workflow

How source code moves through the MiniLex pipeline

1 → 2 → 3 → 4 → 5

1 Source Code

Enter or upload a sample program

2 Token Rules

Apply regular-expression patterns

3 Lexical Analysis

Scan input and identify lexemes

4 Automata

Generate NFA and convert it to DFA

5 Results

Display tokens, errors and state transitions

The output provides a traceable view of how each input lexeme is recognized or rejected.

Token Categories

Token rules supported by the MiniLex workbench

Keywords

int, float, if, else, return

Identifiers

Variable and function names

Numeric Constants

Integer and floating-point values

Operators

+, -, *, /, =, ==, <, &&

Delimiters

; , ( ) { }

Comments & Strings

Single-line comments, block comments and string literals

Lexical Analyzer Engine

Core processing performed on the input program

- Reads source code character by character and groups characters into meaningful lexemes.
- Recognizes token types using configurable regular-expression rules.
- Uses longest-match behavior to select the longest valid lexeme.
- Tracks line and column positions for every token.
- Handles whitespace and comments while preserving source locations.
- Reports malformed lexemes and invalid characters instead of silently ignoring them.

NFA and DFA Generation

Connecting regular-expression rules to finite automata

REGEX → NFA

- Use Thompson's construction to build an NFA from a regular expression.
- Represent start states, accepting states, transitions and epsilon transitions.
- Display the generated NFA as an interactive state diagram.

NFA → DFA

- Calculate epsilon closures and apply subset construction.
- Generate DFA states and transitions from NFA state sets.
- Compare NFA and DFA transition tables and simulate input acceptance.

Analysis Results and Error Reporting

What users can inspect after running lexical analysis

- Token table: token type, lexeme, line number and column number.
- State trace: current state, input character, transition and final decision.
- Lexical errors: error category, offending lexeme and exact source location.
- Token statistics: counts and distribution of recognized token categories.
- Export options: downloadable CSV, JSON and TXT analysis reports.

Testing and Expected Deliverables

Validation of the implementation against the PS-01 requirements

- Test keyword, identifier, numeric constant, operator and delimiter recognition.
- Test comments, whitespace, multi-line input and longest-match behavior.
- Test invalid characters, malformed numbers and unterminated strings or comments.
- Verify NFA construction, epsilon closure, DFA conversion and lexeme simulation.
- Deliver a working web application, modular lexer and automata engine, test suite and project documentation.

Conclusion

MiniLex: making lexical analysis easier to understand

MiniLex connects compiler theory with practical implementation by turning token recognition and finite automata into an interactive learning experience.

Thank You

Guide: UMA Shankari | Team: Allipuram Akhil, B. Harneesh Yadav, A. Dhanush